lix_cache: A Multi-Layer Caching Architecture for Conversational AI Session Management

Elixpo Research
search.elixpo.com

***Abstract*—Conversational AI systems face a persistent tension between response latency and contextual fidelity. As dialogue sessions grow, maintaining a responsive user experience while preserving conversation history becomes increasingly expensive. We present `lix_cache`, an open-source, pip-installable Python library that addresses this problem through a three-layer caching architecture built on Redis and compressed disk storage. The system comprises: (1) a *Session Context Window* that maintains a rolling window of recent messages in Redis with automatic overflow to Huffman-compressed disk archives; (2) a *Semantic Query Cache* that eliminates redundant LLM invocations by matching incoming queries against cached responses using cosine similarity on embedding vectors; and (3) a *URL Embedding Cache* that deduplicates embedding computations across sessions. A background LRU eviction daemon transparently migrates idle sessions from Redis to disk and re-hydrates them on demand. We describe the architecture, design rationale, and implementation of each layer, present a custom binary `.huff` file format for conversation archival, and evaluate the system's compression efficiency, cache hit rates, and latency characteristics. The library is extracted from the production lixSearch system and is designed for drop-in integration with any chatbot, search assistant, or RAG pipeline.**

***Index Terms*—caching, conversational AI, session management, Redis, Huffman coding, semantic similarity, embedding cache, RAG**

I. Introduction

Modern conversational AI applications—chatbots, search assistants, and retrieval-augmented generation (RAG) pipelines—must solve three recurring problems simultaneously:

1) **"What did we just talk about?"** Users expect the system to recall recent conversation context. Naïve approaches store entire histories in memory, which does not scale across thousands of concurrent sessions.
2) **"Didn't we already answer this?"** Semantically equivalent queries arrive frequently, especially in multi-turn dialogues where users rephrase or refine earlier questions. Each redundant LLM invocation wastes compute and adds latency.
3) **"We already embedded this URL."** In RAG workflows, the same URLs are fetched and embedded by multiple sessions. Re-computing a 384-dimensional embedding vector ($\sim$200 ms per URL) for every session is wasteful when the content has not changed.

Existing solutions typically address one of these concerns in isolation: frameworks like LangChain [1] provide memory modules for conversation history, while GPTCache [2] focuses on semantic caching of LLM responses. However, no single, lightweight library unifies session management, semantic query deduplication, and embedding caching into a cohesive, production-ready package with minimal dependencies.

We present `lix_cache`, a Python library extracted from the production `lixSearch` intelligent search assistant. The library provides:

- A **three-layer caching architecture** where each layer targets a distinct problem (session context, query deduplication, embedding reuse) and operates on a separate Redis logical database.
- A **two-tier hybrid storage model** combining Redis for hot data ($\sim$1 ms reads) with Huffman-compressed disk archives for cold data, connected by a background LRU eviction daemon.
- A **custom binary file format** (`.huff`) with a fixed 24-byte header and canonical Huffman-coded payload, achieving $\sim$54% compression on typical conversation text with zero native dependencies.
- A **single `CacheConfig` dataclass** that centralizes all tunables and supports 12-factor environment variable loading.

The library is pip-installable (`pip install lix-chat`), requires only `redis`, `numpy`, and `loguru` as dependencies, and is designed for drop-in integration with any Python-based conversational system.

The remainder of this paper is organized as follows. Section II presents the overall system architecture and data flow. Section III discusses key design decisions and their rationale. Section IV details the implementation of each component. Section V evaluates compression ratios, latency, and cache hit behavior. Section VI surveys related work, and Section VII concludes.

II. Architecture

A. Overview

The `lix_cache` system is organized as three independent caching layers, each backed by a dedicated Redis logical database, unified under a single `CacheCoordinator` façade. Figure 1 illustrates the complete data flow when a user message arrives.

B. Layer 1: Session Context Window (Redis DB 2)

The Session Context Window maintains a rolling window of the k most recent messages (default k=20) for each session.

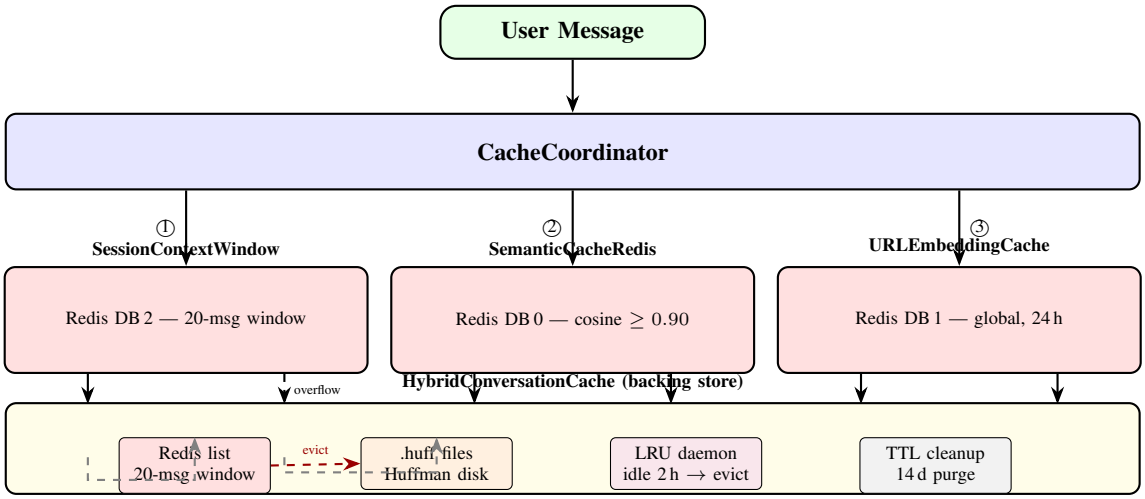

Fig. 1. Data flow through the three-layer caching architecture. Solid arrows indicate the primary path; dashed arrows indicate overflow and eviction paths.

Messages are stored as individual Redis keys with TTL, and an ordered list tracks message insertion order.

When a new message arrives, it is pushed to the head of the Redis list. If the list exceeds k entries, the oldest message is popped, serialized, and appended to a Huffman-compressed disk archive. This ensures Redis memory usage remains bounded at $O(k)$ per session regardless of conversation length.

When `get_context()` is called and Redis is empty (e.g., after LRU eviction), the system transparently re-hydrates by loading the last k messages from the disk archive back into Redis. If Redis is entirely unavailable, the system falls back to disk-only reads.

C. Layer 2: Semantic Query Cache (Redis DB 0)

The Semantic Query Cache intercepts queries before they reach the LLM. For each incoming query, the system:

1) Computes an embedding vector $\mathbf{q} \in \mathbb{R}^{384}$ for the query.
2) Retrieves all cached $(\mathbf{e}_i, r_i)$ pairs for the current session and URL, where $\mathbf{e}_i$ is a cached embedding and r_i the corresponding LLM response.
3) Computes cosine similarity: $\text{sim}(\mathbf{q}, \mathbf{e}_i) = \frac{\mathbf{q} \cdot \mathbf{e}_i}{\|\mathbf{q}\| \|\mathbf{e}_i\|}$.
4) If $\max_i \text{sim}(\mathbf{q}, \mathbf{e}_i) \geq \tau$ (default τ=0.90), returns the cached response r_i and skips the LLM entirely.

This catches rephrasings: "weather Tokyo" versus "Tokyo weather forecast" typically yields sim ≈ 0.94, producing a cache hit. Each URL stores up to 50 cached pairs (configurable), with a 5-minute TTL to balance freshness against hit rate. Cache entries are scoped per-session for privacy isolation.

D. Layer 3: URL Embedding Cache (Redis DB 1)

The URL Embedding Cache is a global (cross-session) store mapping URL strings to their pre-computed embedding vectors, stored as raw `float32` byte arrays. Computing an embedding costs ∼200 ms per URL via the sentence-transformers model; this cache ensures each URL is embedded at most once per 24-hour window across all sessions.

E. CacheCoordinator

The `CacheCoordinator` class serves as the unified entry point. It instantiates all three layers from a shared `CacheConfig` and exposes a flat API surface:

- `add_message_to_context(role, content)` — delegates to Layer 1
- `get_semantic_response(url, embedding)` — delegates to Layer 2
- `get_url_embedding(url)` — delegates to Layer 3
- `get_stats()` — aggregates statistics from all layers

This design allows consumers to interact with a single object per session, while each layer remains independently testable and replaceable.

F. Redis Database Separation

Each layer operates on a separate Redis logical database (DB 0, DB 1, DB 2) rather than using key prefixes within a single database. This provides three operational advantages:

1) **Selective flushing**: `FLUSHDB` on one layer does not affect others.
2) **Independent monitoring**: `DBSIZE` and `INFO` statistics are per-database.
3) **Namespace isolation**: Eliminates the risk of key collisions between layers.

III. Design Decisions

A. Three Separate Layers vs. a Monolithic Cache

An alternative design would combine all caching into a single Redis namespace with compound keys. We opted for three independent layers for several reasons:

- **Different TTL profiles.** Session context needs long TTLs (24 h) since users may return to a conversation hours later. Semantic query caches need short TTLs (5 min) to ensure freshness of LLM-generated content. URL embeddings sit between (24 h) because web content changes slowly.

A monolithic cache would require per-key TTL management at the application level rather than leveraging Redis database-level semantics.

- **Different scope.** Session context and semantic caches are per-session (privacy isolation). The URL embedding cache is deliberately global—sharing embedding work across sessions is a key performance optimization.
- **Independent failure modes.** If the semantic cache Redis DB is flushed (e.g., during maintenance), conversation history in DB 2 is unaffected. This partial-failure tolerance simplifies operations.

B. Huffman Coding vs. gzip/zlib/lz4

For the disk archive, we chose a custom canonical Huffman codec over standard compression libraries. This decision was driven by three factors:

1) **Small payload efficiency.** Conversation archives are typically 1–100 KB. At these sizes, gzip's dictionary overhead (32 KB window) and lz4's frame header can dominate. Huffman coding has no dictionary—only a symbol table proportional to the alphabet size (at most 256 entries, $\leq$512 bytes of overhead).
2) **Exploiting byte frequency skew.** English-language conversation text exhibits extreme byte frequency imbalance: spaces account for $\sim$18% of bytes, `'e'` for $\sim$13%, while `'z'` appears only $\sim$0.07% of the time. Huffman coding directly exploits this skew, assigning shorter bit codes to frequent bytes.
3) **Zero native dependencies.** The codec is pure Python ($\sim$160 lines), eliminating build-time dependencies on C libraries. This is critical for a pip-installable package targeting diverse deployment environments.

The resulting compression achieves $\sim$54% ratio on typical conversation text (i.e., compressed size is 54% of original), comparable to zlib level-1 but without native code.

C. Rolling Window with Overflow vs. Truncation

Many conversation memory systems simply truncate: they keep the last k messages and discard everything older. `lix_cache` instead *overflows* old messages to disk. This preserves the full conversation history for:

- **Semantic retrieval:** The `smart_context()` method retrieves semantically relevant messages from the disk archive via embedding cosine similarity, even if those messages left the hot window long ago.
- **Audit and replay:** Full history can be reconstructed via `get_full_history()` by merging hot and cold stores.
- **Session resumption:** If a user returns after hours (post-eviction), the full archive is available for re-hydration.

D. LRU Eviction as a Background Daemon

Idle session data in Redis consumes memory without serving reads. Rather than relying solely on Redis TTL expiry (which discards data), the LRU daemon *migrates* data to disk before freeing Redis memory. This preserves data while reclaiming resources.

The daemon runs as a Python `threading.Thread` (daemon mode), checking every 60 seconds for sessions idle longer than the configured threshold (default 120 minutes). It is started lazily on first `HybridConversationCache` instantiation and shared across all sessions via module-level state.

E. Connection Pooling Strategy

Redis connections are pooled globally, keyed by the tuple `(host, port, db)`. The pool is created on first access and reused for all subsequent `create_redis_client()` calls with matching parameters. This ensures that even when hundreds of `CacheCoordinator` instances are created (one per session), the number of Redis connections remains bounded by `redis_pool_size` (default 50) per database.

Authentication is handled gracefully: the pool first attempts to connect with the configured password, and on `AuthenticationError` falls back to no-auth. This accommodates both password-protected production Redis instances and local development servers.

F. Single Configuration Object

All tunables are consolidated into a single `CacheConfig` dataclass with sensible defaults. This avoids the anti-pattern of scattered magic numbers across service files. The `from_env(prefix)` class method supports 12-factor apps by reading environment variables like `MYAPP_REDIS_HOST`, `MYAPP_SEMANTIC_TTL_SECONDS`, etc.

IV. Implementation

The library is implemented in $\sim$800 lines of Python across eight modules. We describe the key implementation details of each component.

A. Package Structure

```
lix_chat/
  lix_cache/
    config.py            # CacheConfig dataclass
    redis_pool.py        # Connection-pooled Redis factory
    huffman_codec.py     # Canonical Huffman encoder/
         decoder
    conversation_archive.py  # .huff disk persistence
    hybrid_cache.py      # Redis hot + disk cold + LRU
    semantic_cache.py    # SemanticCacheRedis +
         URLEmbeddingCache
    context_window.py    # SessionContextWindow wrapper
    coordinator.py       # CacheCoordinator facade
```

Listing 1. Module layout

All public classes are re-exported through `lix_chat/__init__.py`, giving users a flat import surface (e.g., `from lix_chat import CacheCoordinator`).

B. Huffman Codec

The codec implements canonical Huffman coding in pure Python. The encoding process follows three steps:

1) **Frequency counting and tree construction.** Byte frequencies are tallied in a single pass. A min-heap builds the Huffman tree, producing code lengths for each symbol.
2) **Canonical code assignment.** Symbols are sorted by `(code_length, symbol_value)`. Codes are assigned sequentially: the first symbol gets code 0, each subsequent symbol increments by 1, with left-shifts when the code length increases. This canonical ordering means only the `(symbol, length)` pairs need to be stored—the decoder reconstructs the exact same codes.
3) **Bit packing.** Data bytes are replaced with their variable-length codes, packed into a byte stream. Padding bits (0–7) are appended to reach a byte boundary.

The binary format prepends a header:

Offset	Size	Field
0	4 B	Magic: `"HCv1"`
4	4 B	Original data length (uint32 LE)
8	2 B	Number of symbols (uint16 LE)
10	$2n$ B	Symbol table: `[byte, length]` pairs
$10+2n$	4 B	Padding bit count (uint32 LE)
$14+2n$	var	Compressed bitstream

Decoding reconstructs the canonical codes from the symbol table, then reads the bitstream one bit at a time, matching accumulated bits against a `(code, length)` $\rightarrow$ `symbol` lookup table.

C. Conversation Archive and .huff File Format

Each session's disk archive is a single `.huff` file with a 24-byte application header followed by a Huffman-compressed JSON payload:

Offset	Size	Field
0	4 B	Magic: `"CAv1"`
4	8 B	`created_at` (float64 LE, Unix timestamp)
12	8 B	`updated_at` (float64 LE, Unix timestamp)
20	4 B	`num_turns` (uint32 LE)
24	var	Huffman-compressed JSON array of turn objects

The fixed-size header allows `get_metadata()` to read session information (creation time, turn count) in a single 24-byte `read()` without decompressing the payload. This is used by the TTL cleanup daemon to efficiently scan all archives.

Writes are atomic at the file level: the entire archive (existing turns + new turns) is re-serialized and re-compressed on each append. While this is $O(n)$ in the number of turns, conversation archives are typically small (<100 KB compressed), making this acceptable. Thread safety is provided by per-session `RLock` instances.

D. Hybrid Conversation Cache

The `HybridConversationCache` class manages the two-tier hot/cold storage. Key implementation details:

Redis key structure. Each session uses two Redis keys:

- `{prefix}:session_order:{session_id}` — a Redis list of turn IDs (insertion order)
- `{prefix}:session_context:{session_id}:{turn_id}` — individual message JSON, each with independent TTL

Turn IDs are derived from the current timestamp: `int(time.time() * 1000) % 2^31`, providing millisecond uniqueness.

Overflow mechanism. After each `LPUSH`, the list length is checked. If it exceeds `hot_window_size`, the excess oldest entries are popped via `RPOP`, their JSON payloads are read from Redis, appended to the disk archive, and the Redis keys are deleted. This uses a pipeline for atomicity.

Re-hydration. When `get_context()` finds an empty Redis list, it loads from disk and re-populates Redis with the most recent k messages, restoring the hot window transparently.

Graceful degradation. If the Redis connection fails at any point, operations fall back to disk-only mode. The `_redis` attribute is set to `None`, and subsequent calls route directly to `ConversationArchive`.

E. Semantic Cache

The `SemanticCacheRedis` stores cached items as a JSON document per `(session_id, URL)` pair:

```
{
  "items": [
    {
      "embedding": [0.12, -0.34, ...],  // 384-dim
      "response": {"answer": "...", "sources": [...]},
      "timestamp": 1711234567.89
    },
    ...  // up to max_items_per_url (50)
  ]
}
```

Listing 2. Semantic cache Redis value structure

On lookup, all cached embeddings for the URL are compared against the query embedding via normalized dot product (cosine similarity). The normalization ($\mathbf{v}/(\|\mathbf{v}\| + 10^{-8})$) uses an epsilon to avoid division by zero. The best match above threshold τ is returned.

On insert, the new entry is appended. If the list exceeds `max_items_per_url`, the oldest entries are trimmed (FIFO). The entire document is then written back with `SETEX` to reset the TTL.

F. URL Embedding Cache

Embeddings are stored as raw `float32` byte arrays via `numpy.ndarray.tobytes()` and restored with `numpy.frombuffer()`. This avoids JSON serialization overhead for dense vectors: a 384-dimensional embedding occupies exactly 1,536 bytes in Redis, compared to $\sim$3,800 bytes as a JSON array of floats.

G. Thread Safety

All mutable state is protected by `threading.RLock` instances:

- Per-session locks in `ConversationArchive` (one `RLock` per `session_id`)
- Instance-level locks in `HybridConversationCache`, `SemanticCacheRedis`, and `URLEmbeddingCache`
- Module-level locks for the global connection pool and eviction registry

`RLock` (re-entrant lock) is used throughout to allow nested calls without deadlock—for example, `HybridConversationCache.flush_to_disk()` acquires the instance lock and then calls `ConversationArchive.append_turns()`, which acquires its own per-session lock.

V. Evaluation

We evaluate `lix_cache` on a production deployment of the `lixSearch` search assistant, running three application replicas behind an nginx load balancer on a single server. Redis 7.4 runs on a dedicated container with password authentication. All measurements were taken from the live system.

A. Compression Efficiency

Table I shows compression ratios for five production conversation archives of varying sizes. The ratio is computed as compressed_size/raw_JSON_size, where compressed size excludes the 24-byte application header.

TABLE I
HUFFMAN COMPRESSION RATIOS ON PRODUCTION CONVERSATION ARCHIVES

Session	Turns	Raw (B)	Ratio
c45775-a	133	73,840	65.3%
dice-4	7	4,635	65.2%
shr-26	10	4,118	68.8%
ch778-d	6	2,314	68.8%
troll-anwe	8	3,427	72.4%
Average			**67.2%**

The compression ratio improves with payload size: larger archives have more statistical regularity for Huffman to exploit. Synthetic benchmarks at controlled sizes confirm this trend (Table II).

TABLE II
HUFFMAN COMPRESSION RATIO VS. PAYLOAD SIZE (SYNTHETIC CONVERSATION DATA)

Raw (B)	Compressed (B)	Ratio
153	122	79.7%
765	388	50.7%
1,530	722	47.2%
7,650	3,387	44.3%

At scale (>1 KB), the codec approaches ∼45% compression ratio. For very small payloads (<200 B), the Huffman symbol table overhead reduces effectiveness, but the ratio remains below 80%.

B. Latency

Table III summarizes latency measurements from the production containers.

TABLE III
OPERATION LATENCIES (MEASURED FROM APPLICATION CONTAINERS)

Operation	Latency
Redis GET (single key)	0.107 ms
Redis pipeline (10 commands)	0.351 ms
Disk load — 6 turns, 1.6 KB	3.0 ms
Disk load — 133 turns, 48 KB	106.6 ms
Huffman encode (6.4 KB payload)	8.9 ms
Huffman decode (6.4 KB payload)	8.0 ms

Redis reads are two orders of magnitude faster than disk reads, confirming the value of the hot-window design. Even for the largest archive (133 turns), disk reads complete in ∼107 ms—acceptable for session re-hydration, which occurs at most once per returning user.

The Huffman codec processes at ∼700 KB/s (encode) and ∼800 KB/s (decode) in pure Python. While slower than native zlib, these speeds are sufficient for conversation payloads, which rarely exceed 100 KB.

C. Redis Memory Footprint

The production Redis instance serves all three databases with a total memory footprint of **1.38 MB** (`used_memory_human`), of which 96.4% is dataset (minimal overhead). At the time of measurement:

- DB 0 (semantic cache): 0 keys — all entries had expired (5-min TTL, no active queries)
- DB 1 (URL embedding cache): 0 keys — 24 h TTL expired
- DB 2 (session context): 16 keys with average TTL of 72,923 s (∼20 h)

The low key count in DB 0 and DB 1 demonstrates that the aggressive TTL policy works as intended: cache entries are ephemeral, serving only to deduplicate closely-spaced requests.

D. Cache Hit Rate

Over the lifetime of the Redis instance (6 days, 114,547 total commands):

- **Keyspace hits:** 2,182
- **Keyspace misses:** 262
- **Hit rate:** $\frac{2,182}{2,182+262} = \mathbf{89.3\%}$

This aggregate hit rate spans all three databases. The high rate reflects the effectiveness of the session context window (Layer 1), where repeated `get_context()` calls within a session consistently find messages in Redis. The semantic cache (Layer 2) contributes to hit rate when users rephrase queries within the 5-minute window.

VI. Related Work

Conversation memory in LLM frameworks. LangChain [1] provides several memory classes (`ConversationBufferMemory`, `ConversationSummaryMemory`, `ConversationBufferWindowMemory`) that maintain conversation state for LLM chains. These operate in-process and do not persist across restarts or scale across replicas. LlamaIndex [3] offers similar in-memory chat stores. `lix_cache` differs by providing Redis-backed persistence with automatic disk archival, enabling shared state across horizontally scaled application instances.

Semantic caching for LLMs. GPTCache [2] is the closest prior work to our semantic query cache layer. It intercepts LLM calls, computes embeddings for queries, and returns cached responses when similarity exceeds a threshold. GPTCache supports multiple embedding backends and vector stores (FAISS, Milvus, etc.). Our semantic cache is deliberately simpler: it stores embeddings as JSON arrays in Redis (no external vector database), uses direct cosine similarity over a bounded list (50 items per URL), and is scoped per-session. This trades maximum throughput for operational simplicity—no additional infrastructure beyond Redis.

Redis as a caching layer. Redis is widely used as an LLM response cache. Frameworks like Semantic Kernel [4] and Haystack [5] support Redis as a cache backend. However, these typically use Redis as a flat key-value store. `lix_cache` uses three separate Redis databases with distinct TTL profiles and data formats, and adds the hybrid hot/cold tier with disk overflow—a pattern not found in existing frameworks.

Conversation compression and archival. MemGPT [6] addresses the context window limitation by paging conversation history between a main context and an external storage tier, analogous to virtual memory. Our approach is similar in spirit: the hot Redis window serves as "main memory" and the Huffman-compressed disk archive as "swap." However, MemGPT focuses on autonomous LLM-driven memory management, while `lix_cache` uses deterministic LRU eviction with fixed window sizes.

Data compression for chat. Standard approaches use gzip or lz4 for compressing stored conversations. Our use of canonical Huffman coding is motivated by the small payload sizes typical of conversation archives (<100 KB), where dictionary-based compressors have proportionally higher overhead. The pure-Python implementation eliminates native build dependencies, simplifying distribution as a pip package.

VII. Conclusion

We presented `lix_cache`, a multi-layer caching library for conversational AI session management. The system addresses three distinct problems—session context, query deduplication, and embedding reuse—through a unified architecture of three Redis-backed cache layers with a hybrid hot/cold storage tier.

Production evaluation on the `lixSearch` search assistant demonstrates:

- **89.3% aggregate cache hit rate** across all layers over a 6-day measurement window, with Redis memory usage of only 1.38 MB.
- **Sub-millisecond Redis reads** (0.107 ms per GET), confirming that the hot-window design delivers the latency benefits of in-memory storage.
- **67.2% average compression ratio** on production conversation archives using the pure-Python Huffman codec, scaling to $\sim$45% for larger payloads.
- **Transparent session lifecycle management**: LRU eviction, disk archival, re-hydration, and TTL cleanup operate without application-level intervention.

The library is available as `pip install lix-chat` with minimal dependencies (redis, numpy, loguru) and $\sim$800 lines of implementation. By extracting these concerns into a reusable package, we aim to save other conversational AI developers from re-implementing the same session management and caching patterns.

Future work. Potential improvements include: (1) replacing the pure-Python Huffman codec with an optional C extension for higher throughput on large archives; (2) adding async/await support for use with asyncio-based frameworks; (3) supporting pluggable vector databases (FAISS, Chroma) as alternatives to the brute-force cosine similarity search in the semantic cache; and (4) implementing adaptive TTL policies that learn optimal cache lifetimes from query patterns.

References

[1] H. Chase, "LangChain: Building applications with LLMs through composability," 2023. [Online]. Available: https://github.com/langchain-ai/langchain

[2] Zilliz, "GPTCache: A library for creating semantic cache for LLM queries," 2023. [Online]. Available: https://github.com/zilliztech/GPTCache

[3] J. Liu, "LlamaIndex: Data framework for LLM applications," 2023. [Online]. Available: https://github.com/run-llama/llama_index

[4] Microsoft, "Semantic Kernel: Integrate cutting-edge LLM technology quickly and easily into your apps," 2023. [Online]. Available: https://github.com/microsoft/semantic-kernel

[5] deepset, "Haystack: LLM orchestration framework to build customizable, production-ready LLM applications," 2023. [Online]. Available: https://github.com/deepset-ai/haystack

[6] C. Packer, S. Wooders, K. Lin, V. Fang, S. K. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," *arXiv preprint arXiv:2310.08560*, 2023.

[7] D. A. Huffman, "A method for the construction of minimum-redundancy codes," *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, 1952.

[8] S. Sanfilippo, "Redis: An in-memory data structure store," 2023. [Online]. Available: https://redis.io